



CHAPTER 12

Terraform Security Best Practices

Protecting secrets, securing state files, and enforcing role-based access control in Terraform-managed infrastructure.



Secrets



State Files



RBAC

What We'll Cover

Terraform has no built-in secrets manager — security has to be engineered in.



12.1

Managing Secrets

Env vars, AWS SSM Parameter Store, and HashiCorp Vault instead of hardcoded credentials.



12.2

Securing State Files

Remote encrypted backends, state locking, and IAM access restrictions.



12.3

Role-Based Access Control

IAM roles over static keys, and scoping what Terraform is allowed to touch.

Terraform Has No Built-In Secret Manager

By default, sensitive values — API keys, passwords, tokens — can end up in:



Plaintext .tf files

Hardcoded credentials committed straight into version control.



State files

Resource attributes — including secrets — are stored unencrypted by default.



CLI logs & output

Console output and CI/CD logs can leak values during apply/plan.

THREE WAYS TO FIX IT



Environment Variables

Simplest — pulled automatically by the AWS provider.



AWS SSM Parameter Store

Managed, encrypted, referenced via data source.



HashiCorp Vault

Dynamic secrets, tightest control, most setup.

Avoid Hardcoding Secrets

```
provider "aws" {  
  access_key = "AKIAEXAMPLE"  
  secret_key = "SECRET12345"  
}
```



Problem

Secrets are stored in plaintext inside the .tf file — anyone with repo access (or a leaked commit) has your AWS keys.

Why this matters for instruction

- Once committed, a secret is compromised permanently
- Rotating the key does not remove it from git history
- State files can re-expose the same values even after cleanup

Use Environment Variables



1 Set environment variables

```
$ export AWS_ACCESS_KEY_ID="AKIAEXAMPLE"  
$ export AWS_SECRET_ACCESS_KEY="SECRET12345"
```

2 Reference them in Terraform — no keys needed in code

```
provider "aws" {  
  region = "us-east-1"  
}
```

Terraform automatically picks up credentials from the environment — nothing to hardcode, nothing to leak.

Instructor note: pair this with a .gitignore for .env / .tfvars files containing local overrides.



Store Secrets in AWS SSM Parameter Store

STEP 1 — Store the secret

```
$ aws ssm put-parameter \  
  --name "db_password" \  
  --value "SuperSecret123" \  
  --type "SecureString"
```

STEP 2 — Retrieve it in Terraform

```
data "aws_ssm_parameter" "db_password" {  
  name = "db_password"  
}
```



Why this is better

SSM SecureString values are encrypted at rest with KMS, access is controlled through IAM policy, and Terraform only ever reads the reference — never a hardcoded value. Rotation happens outside your codebase.

Use HashiCorp Vault for Advanced Secret Management



Vault stores secrets encrypted and can generate them dynamically on demand — the strongest option of the three.

STEP 1 — Store a secret in Vault

```
$ vault kv put secret/aws \  
  access_key="AKIAEXAMPLE" \  
  secret_key="SECRET12345"
```

STEP 2 — Retrieve it in Terraform

```
data "vault_generic_secret" "aws" {  
  path = "secret/aws"  
}
```

```
provider "aws" {  
  access_key = data.vault_generic_secret.aws.data["access_key"]  
  secret_key = data.vault_generic_secret.aws.data["secret_key"]  
}
```



Your State File Knows Everything

terraform.tfstate records the full resource graph — including attributes that were marked sensitive in configuration.



Local State File

- Lives unencrypted on disk
- No locking — concurrent applies corrupt it
- Easy to accidentally commit to git



Remote Encrypted Backend

- Encrypted at rest (e.g. S3 + KMS)
- Locking via DynamoDB prevents races
- Access controlled centrally via IAM



Remote State: S3 + DynamoDB Locking

1 Create the S3 bucket

```
$ aws s3 mb s3://my-terraform-state
$ aws s3api put-bucket-versioning \
  --bucket my-terraform-state \
  --versioning-configuration Status=Enabled
```

2 Enable locking with DynamoDB

```
$ aws dynamodb create-table \
  --table-name terraform-lock \
  --attribute-definitions \
    AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH
```

STEP 3 — Configure the Terraform backend

```
terraform {
  backend "s3" {
    bucket      = "my-terraform-state"
    key         = "terraform.tfstate"
    region     = "us-east-1"
    encrypt     = true
    dynamodb_table = "terraform-lock"
  }
}
```

Restrict State Access with IAM



```
{
  "Effect": "Deny",
  "Action": "s3:GetObject",
  "Resource":
    "arn:aws:s3:::my-terraform-state/terraform.tfstate",
  "Condition": {
    "StringNotEqualsIfExists": {
      "aws:username": "terraform-user"
    }
  }
}
```

Reading the policy

- Denies s3:GetObject on the state object by default
- The Condition carves out an exception for one IAM identity
- Everyone else is blocked from reading the raw state — even though it's in a shared bucket
- Pair with bucket policies + least-privilege roles for defense in depth



Use IAM Roles, Not Static Credentials

STEP 1 — Create the role

```
$ aws iam create-role \  
  --role-name TerraformRole \  
  --assume-role-policy-document \  
    file://trust-policy.json
```

STEP 2 — Assume it in Terraform

```
provider "aws" {  
  region = "us-east-1"  
  assume_role {  
    role_arn =  
      "arn:aws:iam::123456789012:role/TerraformRole"  
  }  
}
```



Result: temporary, auto-expiring credentials

No long-lived access keys to rotate, leak, or accidentally commit — STS issues short-lived tokens for each Terraform run instead.



Scope What Terraform Is Allowed to Touch

Example — allow EC2 lifecycle actions only; everything else (S3, RDS, IAM, ...) stays denied by default.

```
{
  "Effect": "Allow",
  "Action": [
    "ec2:RunInstances",
    "ec2:TerminateInstances",
    "ec2:DescribeInstances"
  ],
  "Resource": "*"
}
```

Why scope by service

- Blast radius shrinks — a compromised run or bad plan can't touch RDS, S3, or IAM
- Codifies least privilege directly alongside infrastructure code
- Pairs naturally with separate roles per environment (dev / staging / prod)

Scenario: Store AWS Credentials Securely



Problem

```
provider "aws" {  
  access_key = "AKIAEXAMPLE"  
  secret_key = "SECRET12345"  
}
```

Exposes sensitive information directly in code.



Solution

```
$ export AWS_ACCESS_KEY_ID="..."  
$ export AWS_SECRET_ACCESS_KEY="..."  
  
provider "aws" {  
  region = "us-east-1"  
}
```

Terraform retrieves credentials from the environment automatically — nothing to leak.

Key Takeaways



Never hardcode secrets in Terraform configuration



Use Vault, AWS SSM, or environment variables for secret management



Secure Terraform state with remote backends and encryption



Restrict access to state files with IAM policies



Use IAM roles instead of static credentials for Terraform execution



Scope Terraform's permissions to only the resources it needs